Topic: Snowflake – Time Travel, Fail-safe

Snowflake Time Travel & Fail-safe

Time Travel Retention by Edition

- **Standard Edition**
    - **Permanent tables: maximum 1 day retention.**
- **Enterprise Edition & Above**
    - **Permanent tables: retention configurable up to 90 days.**

Fail-safe

- Fixed 7-day period for permanent tables.
- Begins immediately after Time Travel ends.
- Data recovery during fail-safe can only be performed by Snowflake support.
- **Maximum total recoverable period:**
    - **90 days (Time Travel) + 7 days (Fail-safe) = 97 days.**

Retention Period Behavior

- **If retention is set to 90 days:**
    - Data stays in Time Travel for 90 days.
    - Moves to Fail-safe from day 91 to day 97.
- **Decreasing retention (e.g., from 10 days to 1 day):**
    - Only 1 day remains in Time Travel.
    - Data from days 2–8 moves to Fail-safe.
    - Data movement happens via a background process, so the change is not immediate.
- **Increasing retention:**
    - Data already in Fail-safe does NOT move back to Time Travel.
    - Newly increased retention fills gradually over time.
- **Once data enters Fail-safe, it never returns to Time Travel.**

Example Clarifications

- **Table created on Oct 1st with 1-day retention:**
    - Older data gradually moves to Fail-safe.
    - By Oct 10th, data from Oct 1st is unrecoverable.

- **Increasing retention from 1 → 3 days on Oct 10th:**
    - Oct 11th: 2 days available
    - Oct 12th: 3 days available
- **Maximum retention cannot exceed 90 days.**

1. Snowflake Time Travel Queries (ALL 3 OPTIONS)

1️⃣ Time Travel Using TIMESTAMP

What is TIMESTAMP-based Time Travel?

TIMESTAMP Time Travel allows you to query a table as it existed at an exact point in time.

You specify a date and time, and Snowflake returns the table state that was valid at that moment.

Think of it as a CCTV snapshot of your data.

How TIMESTAMP Works Internally

- Snowflake stores **versioned micro-partitions**
- Each change is tracked with a **system timestamp**
- When a TIMESTAMP query is issued:
    - Snowflake locates the closest snapshot **≤ given timestamp**
- No physical data copy occurs

📌 Important:

- Timestamp must fall **within the Time Travel retention window**
- Snowflake resolves time using **account time zone**

Syntax

```
SELECT *
FROM table_name
AT (TIMESTAMP => 'YYYY-MM-DD HH24:MI:SS');
```

WHEN Do We Use TIMESTAMP?

✓ Audit corrections
✓ Production data corruption
✓ Accidental DELETE
✓ Wrong UPDATE
✓ Regulatory reporting

Best Use Cases

Scenario	Why TIMESTAMP Works
Audit / compliance	Exact historical point
Regulatory reporting	Data as of cutoff
Daily reconciliation	End-of-day snapshot
Root cause analysis	Known failure time

📌 Use TIMESTAMP when:

- Exact time of issue is **known**
- You need **point-in-time accuracy**

TIMESTAMP Timeline Concept

Past ----------|----------> Future

^ Exact time requested

Snowflake returns data **as it existed just before or at** that timestamp.

We'll answer **3 questions** clearly:

1. **What problem are we solving?**
2. **What does each query actually do?**
3. **Why two steps (backup + restore)?**

1️⃣ **What Problem Are We Solving?**

👉 You **changed or deleted data by mistake**
👉 You know **exact date & time** when data was correct
👉 You want to **go back to that exact moment**

This is called **point-in-time recovery**

2️⃣ **Understand the Situation with a Timeline**

Assume this happened:

10:00 AM → Correct data

10:10 AM → Wrong UPDATE / DELETE

10:20 AM → You realized the mistake

You want data **as it was at 10:00 AM**

Correct Recovery Time

✅ **Any time BEFORE 10:10 AM**
Examples:

- 10:09:59 AM ✅
- 10:05:00 AM ✅
- 10:00:00 AM ✅

Step 1: Create Backup Table (SAFE STEP)

```
CREATE OR REPLACE TABLE products_backup AS
SELECT *
FROM products
AT (TIMESTAMP => '2025-12-09 10:05:00');
```

🔍 **What this query ACTUALLY does**

- Snowflake goes back in time to **10:05 AM**
- Reads how the products table looked **at that moment**
- Creates a **new table** called products_backup
- Original products table is **NOT touched**

📌 Think like:

"Copy the table as it existed at 10:05 AM into a new table"

Step 2: Restore Data (Overwrite Current Data)

```
INSERT OVERWRITE INTO products
SELECT *
FROM products
AT (TIMESTAMP => '2025-12-09 10:05:00');
```

🔍 **What this query ACTUALLY does**

- Deletes **current data** in products
- Replaces it with data from **10:05 AM**
- Table structure stays same
- Only data is replaced

📌 Think like:

"Reset the table back to how it was at 10:05 AM"

Visual Representation (Very Important)

Before Recovery

products

Wrong data ❌

After Step 1

products products_backup

--- ---

Wrong data ❌ Correct data ✅

After Step 2

products products_backup

--- ---

Correct data ✅ Correct data ✅

Recover Data Using TIMESTAMP

Create Backup Table

```
CREATE OR REPLACE TABLE products_backup AS
SELECT *
FROM products
AT (TIMESTAMP => '2025-12-09 10:15:00');
```

Restore Data

```
INSERT OVERWRITE INTO products
SELECT *
FROM products
AT (TIMESTAMP => '2025-12-09 10:15:00');
```

📌 Useful for **point-in-time recovery**

📌 **Explanation**

- Returns table data exactly as it was at the given timestamp.
- Commonly used for **audits and investigations**.

🔑 **Key Takeaway**

- AT (TIMESTAMP) → **Reads past data**
- CREATE TABLE AS SELECT → **Safe copy**
- INSERT OVERWRITE → **Actual restore**
- This is **point-in-time recovery**

2️⃣ Time Travel Using OFFSET

What is OFFSET-based Time Travel?

OFFSET Time Travel allows you to query a table as it existed a specific number of seconds in the past, relative to the current system time.

Instead of referencing:

- an exact timestamp or
- a specific SQL statement,

you simply say:

"Show me the table as it was *N seconds ago*."

How OFFSET Works Internally

- OFFSET is calculated from **current time (CURRENT_TIMESTAMP)**
- Snowflake moves **backward in seconds**
- OFFSET uses **micro-partition metadata**
- No data copy happens

📌 Key detail:

- OFFSET is always **negative**
- Unit = **seconds only**

Use case

Go back in time by **seconds** from the current time.

Syntax

SELECT *

FROM table_name

AT (OFFSET => -seconds);

WHEN Do We Use OFFSET?

✅ We use OFFSET when:

✓ Exact timestamp is unknown
✓ We only know how long ago the data was correct
✓ A quick rollback is required
✓ The issue happened recently
✓ We don't have the Query ID (STATEMENT not possible)
✓ In development / testing environments
✓ For debugging recent changes

Best Use Cases

Scenario	**Why OFFSET Works**
Recent accidental update	Time unknown but recent
Quick rollback	Fast recovery
Development / testing	Easy experimentation
Debugging	Check previous state

📌 OFFSET is best when:

- You know **how long ago** something happened
- You do **not know exact timestamp**
- No need for exact statement precision

OFFSET Timeline Concept

NOW ----------> Past

-60 -300 -3600

1 min 5 min 1 hour

Example:

- OFFSET = -300 → 5 minutes ago
- OFFSET = -3600 → 1 hour ago

We'll answer **3 questions clearly**:

1. **What problem are we solving?**
2. **What does each query actually do?**
3. **Why two steps (backup + restore)?**

1️⃣ What Problem Are We Solving?

👉 You changed or deleted data by mistake
👉 You **do NOT know exact timestamp**, but you know **how long ago** the data was correct
👉 You want to go back to that previous state

This is called **relative point-in-time recovery**

2️⃣ Understand the Situation with a Timeline

Assume this happened:

10:00 AM → Correct data

10:10 AM → Wrong UPDATE / DELETE

10:20 AM → You realized the mistake (NOW)

You want data **as it was before 10:10 AM**.

Current time = **10:20 AM**

3️⃣ Calculate the Correct OFFSET

OFFSET always works as:

CURRENT TIME – OFFSET (in seconds)

To go **before 10:10 AM**, we must go back **more than 10 minutes**.

Step 1: Create Backup Table (SAFE STEP)

```
CREATE OR REPLACE TABLE products_backup AS
SELECT *
FROM products
AT (OFFSET => -900);
```

🔍 **What this query ACTUALLY does**

- Snowflake goes back **15 minutes from NOW**
- Reads how the products table looked at that time
- Creates a new table called products_backup
- Original products table is **NOT touched**

📌 Think like:

“Copy the table as it existed 15 minutes ago into a new table”

Step 2: Restore Data (Overwrite Current Data)

```
INSERT OVERWRITE INTO products
SELECT *
FROM products
AT (OFFSET => -900);
```

What this query ACTUALLY does

- Deletes **current data** in products
- Replaces it with data from **15 minutes ago**
- Table structure remains unchanged
- Only the data is restored

📌 Think like:

“Reset the table back to how it was 15 minutes ago”

Visual Representation (Very Important)

Before Recovery

products

Wrong data ❌

After Step 1

products products_backup

--- ---

Wrong data ❌ Correct data ✅

After Step 2

products products_backup

--- ---

Correct data ✅ Correct data ✅

Recover Data Using OFFSET

Create backup

```
CREATE OR REPLACE TABLE employees_backup AS
SELECT *
FROM employees
AT (OFFSET => -300);
```

Restore data

```
INSERT OVERWRITE INTO employees
SELECT *
FROM employees
AT (OFFSET => -300);
```

📌 OFFSET gives **state-based recovery**, not row-level undo

Example

Get data from **5 minutes ago**:

```
SELECT * FROM customers AT (OFFSET => -300);
```

📌 **Explanation**

- Offset is always in **seconds**
- Negative value means “go back”
- Useful when exact timestamp is unknown

Real-World Scenario

Situation:
Developer ran wrong UPDATE 5 minutes ago.

Solution:

CREATE TABLE emp_restore AS

SELECT *

FROM employees

AT (OFFSET => -300);

✓ Fast
✓ No query ID needed
✓ Works in emergencies

3️⃣ Time Travel Using STATEMENT

What is STATEMENT-based Time Travel?

STATEMENT Time Travel allows you to query a table as it existed immediately before or after a specific SQL statement was executed.

Instead of thinking in terms of time, you think in terms of an action (UPDATE, DELETE, MERGE, INSERT, TRUNCATE).

This makes it extremely precise for recovery.

How STATEMENT Time Travel Works Internally

When any DML or DDL statement runs in Snowflake:

- Snowflake generates a **unique Query ID**
- That Query ID represents a **single atomic change**
- Snowflake keeps metadata snapshots tied to that Query ID
- Time Travel lets you jump to:
    - The table **state before that statement**
    - The table **state after that statement**

📌 Important:

- Snowflake does **not copy full data**
- It stores **micro-partition metadata**
- This makes STATEMENT travel fast and storage-efficient

We'll answer 3 questions clearly:

1. What problem are we solving?
2. What does each query actually do?
3. Why two steps (backup + restore)?

1️⃣ What Problem Are We Solving?

👉 You changed or deleted data by mistake
👉 You do NOT want to guess time or seconds
👉 You know exact SQL statement that caused the issue

This is called exact-state recovery

2️⃣ Understand the Situation with a Timeline

Assume this happened:

10:00 AM → Correct data

10:10 AM → Wrong UPDATE / DELETE (Query ID = Q123)

10:20 AM → You realized the mistake

You want data as it was just BEFORE the wrong SQL ran.

3️⃣ Identify the Query ID (VERY IMPORTANT)

SELECT LAST_QUERY_ID();

OR from Query History UI.

Assume:

Query ID = Q123

📌 This Query ID uniquely represents the bad SQL statement.

Step 1: Create Backup Table (SAFE STEP)

```
CREATE OR REPLACE TABLE products_backup AS
SELECT *
FROM products
BEFORE (STATEMENT => 'Q123');
```

🔍 What this query ACTUALLY does

- Snowflake finds the exact moment **before Query Q123**
- Reads how the products table looked at that instant
- Creates a new table called products_backup
- Original products table is **NOT touched**

📌 Think like:

“Copy the table as it existed just before the bad SQL ran”

Step 2: Restore Data (Overwrite Current Data)

```
INSERT OVERWRITE INTO products
SELECT *
FROM products
BEFORE (STATEMENT => 'Q123');
```

🔍 **What this query ACTUALLY does**

- Deletes **current (wrong) data** in products
- Replaces it with data from **just before Query Q123**
- Table structure stays the same
- Only the data is restored

📌 Think like:

“Reset the table back to its state before the bad SQL”

Visual Representation (Very Important)

Before Recovery

products

Wrong data ❌

After Step 1

products products_backup

--- ---

Wrong data ❌ Correct data ✅

After Step 2

products products_backup

--- ---

Correct data ✅ Correct data ✅

AFTER vs BEFORE (STATEMENT) – Key Difference

BEFORE

BEFORE (STATEMENT => 'Q123')

✓ State **before mistake**
✓ Used for recovery

AFTER

AFTER (STATEMENT => 'Q123')

✓ State **after statement execution**
✓ Used for auditing

📌 **Recovery always uses BEFORE**

When EXACTLY Do We Use STATEMENT?

✓ Production incidents
✓ Accidental DELETE / UPDATE
✓ MERGE failures
✓ Compliance & auditing
✓ Zero-guess recovery

🔑 **Key Takeaway (VERY IMPORTANT)**

- BEFORE (STATEMENT) → Correct recovery point
- AFTER (STATEMENT) → Audit comparison
- No time guessing
- Most accurate method

🏆 Final Ranking (Recovery Safety)

1️⃣ **STATEMENT** – safest
2️⃣ **TIMESTAMP** – accurate
3️⃣ **OFFSET** – quick but risky

2. Restore / Undrop Using Time Travel

Recover Dropped Table

DROP TABLE customers;

Recover it:

UNDROP TABLE customers;

📌 Works only within **Time Travel period**

Restore Table to Previous State

CREATE OR REPLACE TABLE customers_restore AS

SELECT *

FROM customers

AT (OFFSET => -600);

📌 Creates a new table with old data

---

3. Retention Period Queries

Check Retention Period

SHOW TABLES LIKE 'CUSTOMERS';

OR

DESC TABLE customers;

Set Retention Period

Set to 1 day

ALTER TABLE customers

SET DATA_RETENTION_TIME_IN_DAYS = 1;

Set to 90 days (Enterprise Edition)

ALTER TABLE customers

SET DATA_RETENTION_TIME_IN_DAYS = 90;

📌 Cannot exceed **90 days**

---

4. Fail-safe (Important Interview Point)

❌ **No SQL queries available for Fail-safe**

📌 Key points to say in interviews:

- Fail-safe is **automatic**
- Lasts **7 days**
- Only **Snowflake Support** can recover data
- Data in Fail-safe **cannot be queried**

---

5. Data Load from CSV (Basic Example)

Create File Format

```
CREATE OR REPLACE FILE FORMAT csv_format
TYPE = 'CSV'
FIELD_DELIMITER = ','
SKIP_HEADER = 1;
```

Load Data

```
COPY INTO customers
FROM @my_stage/customers.csv
FILE_FORMAT = csv_format;
```

---

6. Common Interview Scenarios (With Queries)

SCENARIO 1: Accidentally Deleted Data

What actually happened?

```
DELETE FROM customers;
```

❌ This removed **all rows** from the customers table
✓ Table still exists (only data is gone)

Step 1️⃣ : Get the Query ID of DELETE

```
SELECT LAST_QUERY_ID();
```

Assume:

Query ID = Q999

This Query ID represents the **DELETE statement**.

Step 2️⃣ : Recover Deleted Data (VIEW OLD DATA)

```
SELECT *
FROM customers
BEFORE (STATEMENT => 'Q999');
```

🔍 **What this query REALLY does**

- Snowflake looks at the table **just before the DELETE ran**
- Returns **all rows that existed before deletion**
- This does **NOT** put data back yet
- It only **shows** the old data

📌 Think like:

“Show me the table as it was before I deleted everything”

Step 3️⃣ : Actually, RESTORE the Data

```
INSERT INTO customers
SELECT *
FROM customers
BEFORE (STATEMENT => 'Q999');
```

🔍 **What this query does**

- Inserts the old rows back into the table
- Table returns to its previous state

✓ **Recovery complete**

==========

SCENARIO 2: Restore Data After Wrong UPDATE

What actually happened?

```
UPDATE customers
SET city = 'UNKNOWN';
```

❌ Data is changed incorrectly
✓ Rows still exist

Step 1️⃣ : Get Query ID of UPDATE

```
SELECT LAST_QUERY_ID();
```

Assume:

Query ID = Q888

Step 2️⃣ : Create Backup Table (SAFE PRACTICE)

CREATE OR REPLACE TABLE customers_backup AS

SELECT *

FROM customers

BEFORE (STATEMENT => 'Q888');

🔍 **What this query REALLY does**

- Reads data **before the wrong UPDATE**
- Copies it into customers_backup
- Original table is **not modified**

📌 Think like:

“Save correct data somewhere safe”

Step 3️⃣ : Restore the Original Table

TRUNCATE TABLE customers; -- remove wrong data

🔍 **Meaning in simple words**

TRUNCATE removes ALL rows from the table instantly, but keeps the table structure.

What TRUNCATE actually does

✓ Deletes **all data (rows)**
✓ Keeps:

- Table
- Columns
- Data types
- Constraints

✓ Faster than DELETE
✓ Uses **no WHERE clause**

INSERT INTO customers -- insert correct data

SELECT * FROM customers_backup;

📌 Think like:

“Empty the table first, then refill it with correct data”

Visual Example

Before TRUNCATE

customers

Wrong data ❌

After TRUNCATE

customers

(empty table)

After INSERT

customers

Correct data ✅

---

7. One-Line Interview Answers

Topic	Answer
Time Travel types	Timestamp, Offset, Statement
Max retention	90 days
Fail-safe duration	7 days
Fail-safe access	Snowflake Support only
Can Fail-safe data return?	❌ No

This post covers **concepts + SQL queries + when to use what** 👇

🔹 **1. Snowflake Time Travel – Overview**

Snowflake Time Travel allows querying **historical data** for:

- Accidental DELETE
- Wrong UPDATE / MERGE
- Data audits
- Point-in-time recovery

⏱️ **Retention limits**

- **Standard Edition** → 1 day
- **Enterprise & above** → Up to 90 days

🔹 **2. Fail-safe – Last Line of Defense**

- Fixed **7 days**
- Starts **after Time Travel ends**
- **Only Snowflake Support** can recover data
- ❌ No SQL queries allowed

📌 **Max recovery window = 90 + 7 = 97 days**

🔹 **3. Three Types of Time Travel in Snowflake**

Snowflake provides **3 ways** to travel back in time:

✅ **TIMESTAMP**

✅ **OFFSET**

✅ **STATEMENT**

Each serves a **different real-world use case**.

🔹 **4. Time Travel Using TIMESTAMP**

📌 Query data as it existed at an **exact date & time**

```
SELECT *
FROM products
AT (TIMESTAMP => '2025-12-09 10:05:00');
```

✅ **When to use TIMESTAMP?**

- Exact time of issue is known
- Audits & compliance
- Regulatory reporting
- End-of-day snapshots

📌 Think: *"I know the exact time"*

🔹 **5. Recover Data Using TIMESTAMP (Point-in-Time Recovery)**

Step 1: Create Backup (Safe)

```
CREATE TABLE products_backup AS
SELECT *
FROM products
AT (TIMESTAMP => '2025-12-09 10:05:00');
```

Step 2: Restore Data

```
TRUNCATE TABLE products;

INSERT INTO products
SELECT * FROM products_backup;
```

✓ Restores data exactly as it was at that time

🔹 **6. Time Travel Using OFFSET**

📌 Query data **N seconds ago** from current time

```
SELECT *
FROM products
AT (OFFSET => -900); -- 15 minutes ago
```

✅ **When to use OFFSET?**

- Exact timestamp is unknown
- You know **how long ago** the data was correct
- Quick rollback needed
- Dev / QA environments

📌 Think: *"I know how long ago"*

🔹 **7. Recover Data Using OFFSET**

Step 1: Backup

```
CREATE TABLE products_backup AS
SELECT *
FROM products
AT (OFFSET => -900);
```

Step 2: Restore

```
TRUNCATE TABLE products;
INSERT INTO products
SELECT * FROM products_backup;
```

⚠️ OFFSET is fast but slightly risky if time is guessed wrong

---

🔹 **8. Time Travel Using STATEMENT (Most Accurate)**

📌 Query data **before or after a specific SQL statement**

```
SELECT *
FROM products
BEFORE (STATEMENT => 'QUERY_ID');
```

✅ **When to use STATEMENT?**

- Exact SQL that caused issue is known
- Production incidents
- Accidental DELETE / UPDATE / MERGE
- Zero-guess recovery

📌 Think: *"I know the exact SQL"*

🔹 9. Recover Data Using STATEMENT (Best Practice)

Step 1: Identify Query ID

```
SELECT LAST_QUERY_ID();
```

Step 2: Backup Correct Data

```
CREATE TABLE products_backup AS
SELECT *
FROM products
BEFORE (STATEMENT => 'QUERY_ID');
```

Step 3: Restore

```
TRUNCATE TABLE products;

INSERT INTO products
SELECT * FROM products_backup;
```

✓ Most reliable recovery method

🔹 10. BEFORE vs AFTER (STATEMENT)

Clause	Usage
BEFORE	Data recovery
AFTER	Auditing & comparison

📌 **Recovery always uses BEFORE**

🔹 11. Accidental DELETE – Interview Scenario

```
DELETE FROM customers;
```

Recover:

```
INSERT INTO customers
SELECT *
FROM customers
BEFORE (STATEMENT => 'QUERY_ID');
```

✓ Table restored successfully

🔹 12. Retention Period Management

```
ALTER TABLE customers
SET DATA_RETENTION_TIME_IN_DAYS = 90;
```

✓ Max = 90 days
✓ Cannot increase beyond this

🔹 13. UNDROPs Using Time Travel

```
DROP TABLE customers;

UNDROP TABLE customers;
```

✓ Works only within Time Travel period

🔹 14. Quick Interview Cheat Sheet

Concept	Answer
Time Travel types	Timestamp, Offset, Statement
Safest recovery	STATEMENT
Fail-safe duration	7 days
Fail-safe access	Snowflake Support
Max retention	90 days

🔹 15. Final Memory Trick 🧠

- **STATEMENT** → I know the SQL
- **TIMESTAMP** → I know the time
- **OFFSET** → I know how long ago